

Masterarbeit

Auf dem Weg zum autonom fahrenden Fahrrad: Eine videogestützte Regelung des Weges mit Webbots

Amine Karabila

Sommersemester 2025

Inhaltsverzeichnis

1	Abstract	4
2	Einleitung	5
2.1	Problemstellung	5
2.2	Theoretischer Hintergrund	5
2.2.1	Anna Ranz (2021) – Hardware-Grundlagen	5
2.2.2	Amine Karabila (2022) – Hinterrad-Antrieb & Elektronik	5
2.2.3	Jonah Zander (2023) – PID-Balanceregung	5
2.2.4	Yasin Giray (2024) – YOLO + MPC-Fahrtrichtung	5
3	Überblick zur angewandten Regelungstechnik	5
3.1	Fahrdynamische Modelle nach Schramm / Hiller / Bardini	5
3.1.1	Einspurmodell (Fahrrad): Lateral-, Roll- und Yaw-Dynamik	5
3.1.2	Zweispurmodell (Referenz PKW): Radlasttransfer, Lateral-/Yaw-Dynamik	5
3.1.3	Gültigkeitsbereiche und Modellvergleich	5
3.2	PID-Regelung – Prinzip und Eignung für das Balance-Problem	5
3.3	MPC-Regelung – Prinzip und Eignung für das Fahrtrichtungsproblem	5
4	Webots-Simulationsumgebung	6
4.1	Überblick	6
4.2	Struktur der Simulationsumgebung	7
4.3	Physikalisches Modell des Fahrrads	7
4.4	Regelung und Steuerung	7
4.5	Experiment-Szenarien	7
5	Kombinierte Regelung	7
5.1	Portierung des Balance-PID (Zander)	7
5.2	Portierung des Fahrtrichtungs-MPC (Giray)	7
5.3	Supervisions-/Koppelkaskade	7
5.4	Failsafe-Strategien	7

6	Ergebnisse	7
6.1	Teststrecken	7
6.1.1	Gerade Strecke mit Störimpuls	7
6.1.2	Slalom-Kurs	7
6.1.3	S-Kurve mit Steigung	7
6.1.4	Kreisfahrt	7
6.2	Balance-Stabilität	7
6.3	Spurtreue & Querabweichung	7
6.4	Rechenzeit & Echtzeitfähigkeit	7
7	Grenzen der Arbeit	7
8	Zukünftige Arbeiten (Sensor-Fusion, Reinforcement Learning, Hardware-in-the-Loop)	7
9	Fazit	7
10	Literaturverzeichnis	7

1 Abstract

Diese Masterarbeit beschäftigt sich mit der Entwicklung einer kombinierten Regelungsstrategie für ein autonom fahrendes, selbstbalancierendes Fahrrad, basierend auf den Ergebnissen dreier vorangegangener Forschungsarbeiten. Ziel dieser Arbeit ist die erfolgreiche Integration einer PID-Balanceregung mit einer MPC-basierten Fahrtrichtungsregelung, die mithilfe eines YOLO-Modells zur visuellen Spurdetektion ergänzt wird. Zur Validierung und Optimierung der kombinierten Regelung wurde die Simulationsumgebung Webots verwendet, wodurch die Robustheit und Funktionalität der entwickelten Regelungsalgorithmen realitätsnah überprüft werden konnten. Im Rahmen der Arbeit wurde eine zweistufige Regelungsarchitektur umgesetzt, welche die präzise Balancierung des Fahrrads mittels eines PID-Reglers und die zuverlässige Spurverfolgung sowie Navigation mithilfe eines MPC-Reglers sicherstellt. Die gewonnenen Simulationsergebnisse bestätigen die Machbarkeit und Effizienz des entwickelten Ansatzes und legen damit eine solide Grundlage für die zukünftige Implementierung der Regelung in einem realen autonomen Fahrrad.

2 Einleitung

2.1 Problemstellung

2.2 Theoretischer Hintergrund

2.2.1 Anna Ranz (2021) – Hardware-Grundlagen

2.2.2 Amine Karabila (2022) – Hinterrad-Antrieb & Elektronik

2.2.3 Jonah Zander (2023) – PID-Balanceregung

2.2.4 Yasin Giray (2024) – YOLO + MPC-Fahrtrichtung

3 Überblick zur angewandten Regelungstechnik

3.1 Fahrdynamische Modelle nach Schramm / Hiller / Bardini

3.1.1 Einspurmodell (Fahrrad): Lateral-, Roll- und Yaw-Dynamik

3.1.2 Zweispurmodell (Referenz PKW): Radlasttransfer, Lateral-/Yaw-Dynamik

3.1.3 Gültigkeitsbereiche und Modellvergleich

3.2 PID-Regelung – Prinzip und Eignung für das Balance-Problem

3.3 MPC-Regelung – Prinzip und Eignung für das Fahrtrichtungsproblem

4 Webots-Simulationsumgebung

4.1 Überblick

Webots ist ein frei verfügbares, plattformunabhängiges Robotiksimulationsprogramm, das seit 1998 von Cyberbotics kontinuierlich weiterentwickelt wird. Es kombiniert eine moderne Qt-Oberfläche mit dem auf ODE basierenden Physik-Kernel und einer OpenGL-basierten Rendering-Engine und ermöglicht so fotorealistische 3D-Szenarien inklusive deterministischer Dynamiksimulation [?]. Die Software stellt eine hierarchische Szenenstruktur bereit: *World-Dateien* (*.wbt) enthalten Knoten für Roboter, Objekte und Umweltparameter; jeder Roboter wiederum kapselt Sensor- und Aktuatorknoten, die über Controller-Prozesse in C/C++, Python, Java, MATLAB oder via ROS-Bridge angesteuert werden können [?]. Ein spezieller *Supervisor-API* erlaubt Eingriffe auf Welt-Ebene (z. B. Reset, Logging, Auto-Evaluation) und bildet damit die Grundlage für reproduzierbare Experimente und Wettbewerbe. Typische Einsatzfelder reichen von der frühphasigen Konstruktionsvalidierung (CAD-Import, Material-/Massenzuweisung) über Regelungs- und KI-Entwicklung bis hin zu cloubasierten Multi-Robot-Simulationen. Studien zeigen zudem, dass Webots dank der offenen Schnittstellen zunehmend als Digital-Twin-Plattform für neuartige Robotertypen – etwa weiche Greifer – genutzt wird, ohne dass Spezialhardware erforderlich ist [?].

Kernmerkmale in Kürze:

- ODE-Physik mit justierbarer Schrittweite; reale Zeit oder beschleunigte Batch-Runs
- Große Sensor-/Aktuatorbibliothek (Kamera, LiDAR, IMU, Propeller u. a.)
- ROS 1/2-Integration (`webots_ros2`), Streaming in Web-Browser, HTML-Export
- Versionskontrolle durch Klartext-Weltdaten und Open-Source-Codebasis

4.2 Struktur der Simulationsumgebung

4.3 Physikalisches Modell des Fahrrads

4.4 Regelung und Steuerung

4.5 Experiment-Szenarien

5 Kombinierte Regelung

5.1 Portierung des Balance-PID (Zander)

5.2 Portierung des Fahrtrichtungs-MPC (Giray)

5.3 Supervisions-/Koppelkaskade

5.4 Failsafe-Strategien

6 Ergebnisse

6.1 Teststrecken

6.1.1 Gerade Strecke mit Störimpuls

6.1.2 Slalom-Kurs

6.1.3 S-Kurve mit Steigung

6.1.4 Kreisfahrt

6.2 Balance-Stabilität

6.3 Spurtreue & Querabweichung

6.4 Rechenzeit & Echtzeitfähigkeit

7 Grenzen der Arbeit

8 Zukünftige Arbeiten (Sensor-Fusion, Reinforcement Learning, Hardware-in-the-Loop)

9 Fazit

10 Literaturverzeichnis

Literatur

[web(2024)] Webots – github repository. <https://github.com/cyberbotics/webots>, 2024. Allgemeine Projektübersicht.